ST. JOSEPH'S
COLLEGE OF ENGINEERING
AND TECHNOLOGY,
- PALAI -
AUTONOMOUS

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CSD415 PROJECT PHASE I

First Review

BrowseBlock

AI Browser Extension for Real-Time Attack Detection

Date : 26-10-2026

Team and Guide Details

- Group No : 5

  Deon George (SJC23CS086)
  Amal Kurian Royce (SJC23CS037)
  Maria Royce (SJC23CS159)
  Melbin K Thomas (SJC23CS164)

- Project Guide

  Prof. Divya Sunny
  Assitant Professor, CSE

Contents

- Abstract
- Introduction
- Problem Statement
- Proposed Solution
- System Architecture
- Functional Requirements
- Non Functional Requirements
- Methodology
- Literature Survey
- Software Requirements
- Conclusion
- References

Abstract

- BrowseBlock is an AI/ML-powered browser security platform, made up of a Chromium browser extension and a React web dashboard, that detects, analyzes, explains and helps prevent browser-based cyber threats in real time.
- It covers ten threat categories: URL and fake QR phishing, XSS, credential harvesting, HTML/JavaScript redirects, clickjacking, cryptojacking, malicious file downloads, email phishing, notification scams and browser extension abuse.
- Lightweight, event-driven screening runs in the extension, while computationally intensive analysis (XGBoost, Random Forest, RAG + LLM, static file analysis) is handled by a FastAPI backend.
- Each result is classified as Safe, Suspicious or High-risk and explained with its evidence; scan history and threat reports are stored in MongoDB for authorized users.

Introduction

- Nowadays, attackers use automated systems to carry out web attacks, so an automated system is needed to monitor, detect, identify and prevent them. This is where BrowseBlock becomes important.
- Phishing sites, malicious scripts, scam pop-ups, unsafe downloads and abusive extensions put everyday browsing at risk for students, employees and organizations.
- Existing tools such as Google Safe Browsing, Microsoft Defender SmartScreen, VirusTotal and email spam filters each address only part of the problem.
- Many users have limited cybersecurity knowledge and need clear warnings, understandable explanations and practical recommendations.
- BrowseBlock aims to provide a unified, explainable and user-friendly security layer for individuals and organizations, without replacing existing antivirus, browser security features or enterprise security solutions.

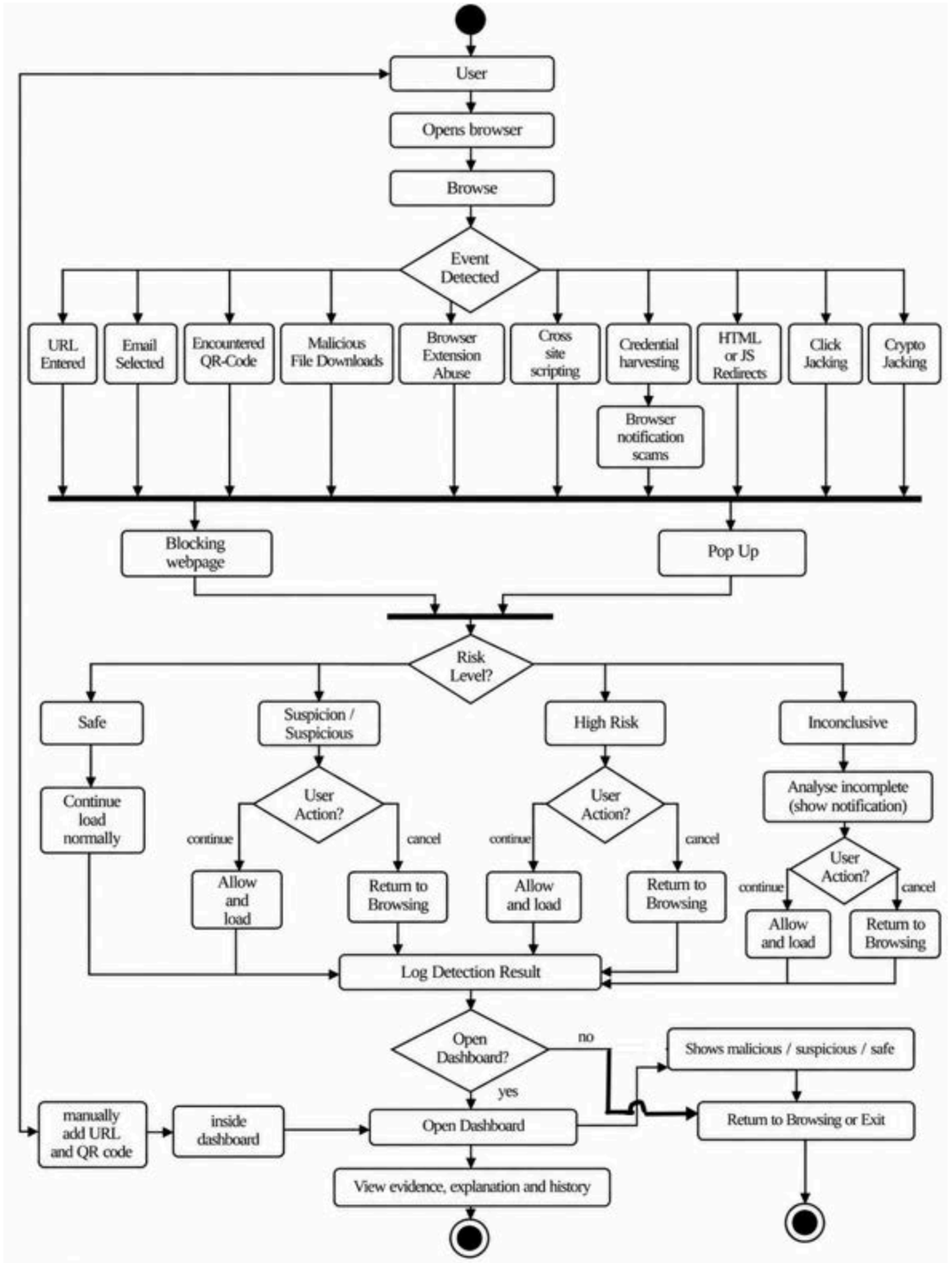
Problem Statement

- Internet users and small organizations lack a single, lightweight and explainable tool that detects and helps prevent the wide range of browser-based attacks (phishing, malicious scripts, scam prompts, unsafe downloads and abusive extensions) in real time, and that clearly tells them why a page, email or file is risky.

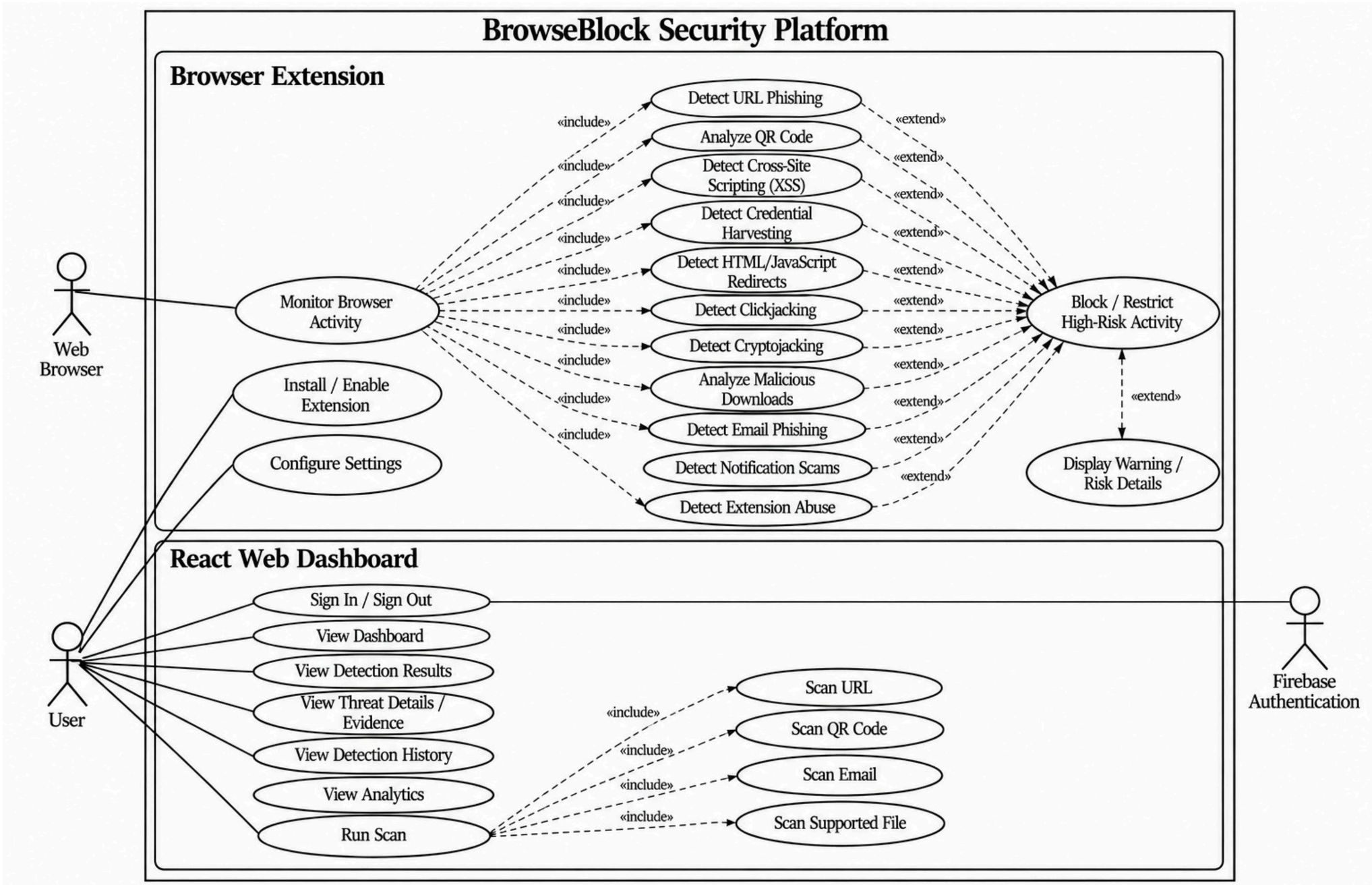
Proposed Solution

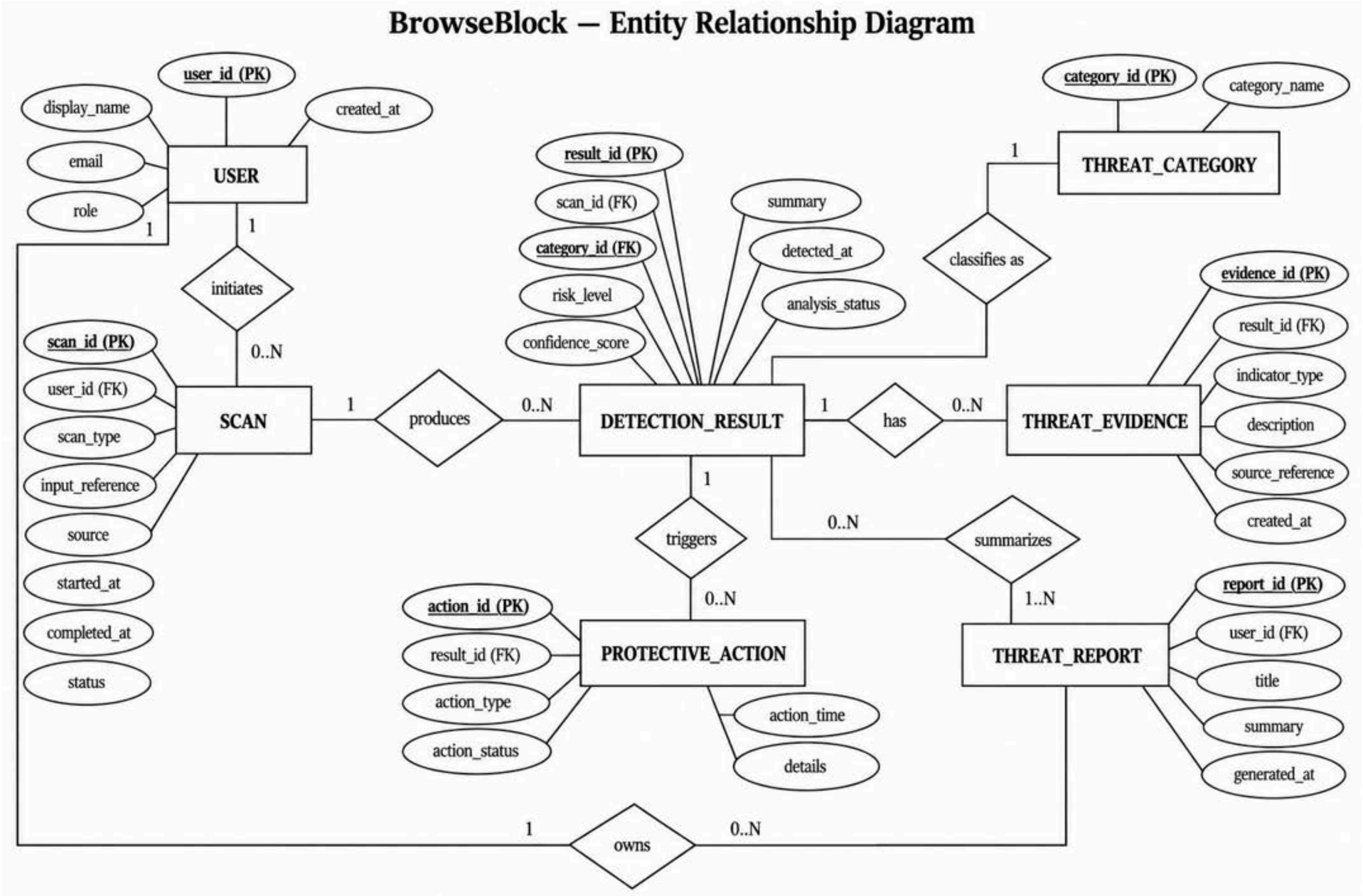
- A Manifest V3 browser extension monitors navigation, page behaviour, redirects and downloads, and warns about or blocks risky activity where browser APIs and permissions allow.
- XGBoost classifies URLs (including URLs decoded from QR codes); Random Forest with SPF/DKIM/DMARC signals screens emails in the extension.
- A RAG + LLM pipeline on the dashboard performs deeper email analysis, and extracted URLs are re-checked by the URL module.
- Downloads (.exe, .zip, .pdf, .doc, .docx) are assessed with static analysis, SHA-256 reputation checks and XGBoost, without executing the files.
- A React dashboard offers manual scans, history, analytics and SHAP-based explanations; Firebase Authentication and MongoDB protect user data.

Activity Diagram

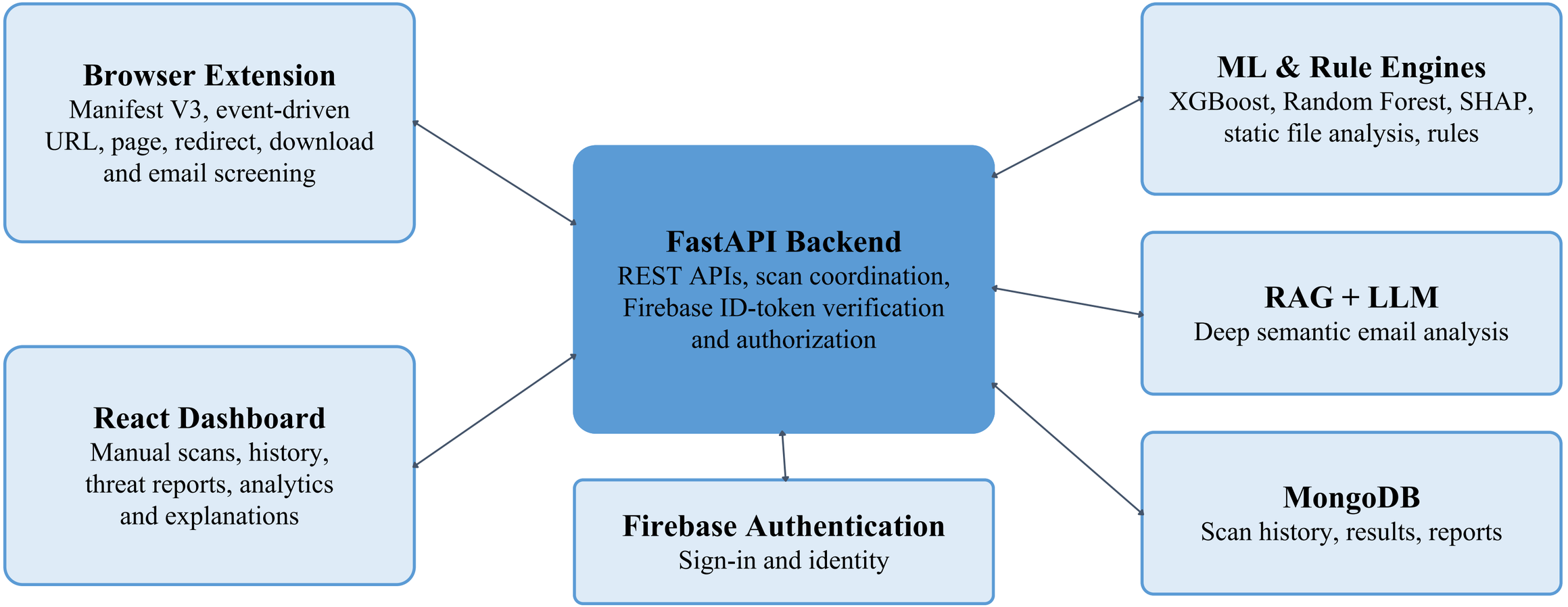


Usecase Diagram




BrowseBlock — Entity Relationship Diagram
user_id (PK)
display_name
created_at
email
USER
role
1
1
initiates
0..N
scan_id (PK)
user_id (FK)
scan_type
input_reference
source
started_at
completed_at
status
SCAN
1
produces
0..N
result_id (PK)
scan_id (FK)
category_id (FK)
risk_level
confidence_score
summary
detected_at
analysis_status
DETECTION_RESULT
1
has
0..N
category_id (PK)
category_name
1
THREAT_CATEGORY
classifies as
evidence_id (PK)
result_id (FK)
indicator_type
description
source_reference
created_at
THREAT_EVIDENCE
1
triggers
0..N
0..N
summarizes
1..N
action_id (PK)
result_id (FK)
action_type
action_status
PROTECTIVE_ACTION
action_time
details
report_id (PK)
user_id (FK)
title
summary
generated_at
THREAT_REPORT
1
owns
0..N

System Architecture



Functional Requirements

- **URL & Fake QR Phishing –** normalize URLs, extract features and classify with XGBoost; decode QR codes and evaluate the embedded URL.
- **Cross-Site Scripting –** detect reflected, stored and DOM-based XSS from script patterns and source-to-sink behaviour; warn and restrict.
- **Credential Harvesting –** monitor external script domains, deceptive logins and scam pop-ups with Playwright-based analysis; block malicious resources.
- **HTML/JavaScript Redirects –** trace HTTP, JavaScript and meta-refresh redirect chains; assess the destination and interrupt malicious navigation.
- **Clickjacking –** inspect iframes, overlays and hidden elements; warn about or restrict risky interactions.

Functional Requirements cont.

- **Cryptojacking –** inspect scripts, WebAssembly, web workers and resource usage; warn about or block mining resources.
- **Malicious File Downloads –** verify file type, analyze EXE/PDF/DOC/DOCX/ZIP statically, hash and check reputation, classify with XGBoost; allow, warn, cancel or quarantine.
- **Email Phishing –** Random Forest and SPF/DKIM/DMARC screening in the extension; RAG + LLM analysis and URL extraction on the dashboard.
- **Notification Scams & Extension Abuse –** detect deceptive notification prompts and abusive extension permissions or script injection; warn the user.
- **Dashboard & Reporting –** Firebase sign-in, manual scans, detection history, threat reports and evidence behind every decision.

Non Functional Requirements

- **Performance :** lightweight, event-driven checks with minimal browsing delay; heavy analysis runs on the backend.
- **Accuracy :** models validated for precision, recall and false-positive rate; model output alone never drives high-impact blocking.
- **Security :** HTTPS, Firebase ID-token verification and ownership/role-based access to MongoDB records.
- **Privacy :** minimal collection and retention of URLs, emails, page content and files; passwords never logged.
- **Safe Processing :** untrusted files are analyzed statically, never executed, within configurable limits.
- **Maintainability :** modular detectors and model versions that can be tested and updated independently.

Methodology

- **Data Preparation:** collect labelled URL, email and file datasets (to be finalized); extract URL/domain, email and file-format features.
- **Modeling:** XGBoost for URL and file risk; Random Forest for email screening; SHAP explanations; RAG + LLM for semantic email analysis.
- **Rule-based Detection:** deterministic rules and DOM/source-sink monitoring for XSS, redirects, clickjacking, cryptojacking, notification scams and extension abuse.
- **Integration:** Manifest V3 extension, FastAPI backend with Firebase token verification, MongoDB storage, React + Tailwind dashboard.
- **Evaluation:** Playwright-based testing; precision, recall and false-positive rates measured on validation data.

Literature Survey

Feature	Existing Systems	Literature Survey	Advantages / Disadvantages	Proposed System
URL / Fake QR Phishing	Google Safe Browsing	XGBoost-Based URL Phishing Detection	Adv: no heavy GPU needed; reported to be more accurate than classical ML such as Random Forest. Dis: multiple datasets needed for training, validation and testing.	XGBoost model for URL (and decoded QR URL) classification.
HTML / JavaScript Redirects	Google Safe Browsing; Chrome webNavigation and declarativeNetRequest	Zeng et al., "Hidden Path: Understanding the Intermediary in Malicious Redirections" (2022)	Adv: reconstructs redirect chains incl. HTTP, JavaScript, HTML, delayed and multi-step redirects. Dis: large-scale crawling is too heavy for an extension; navigation monitoring, reputation checks, risk scoring and blocking are still needed.	Reconstruct HTTP, JS, HTML, delayed and multi-step redirects; webNavigation for monitoring, URL reputation for risk, declarativeNetRequest for blocking.
Cross-Site Scripting (XSS)	NoScript; Content Security Policy (CSP)	Pan & Mao, "Detecting DOM-Sourced Cross-Site Scripting in Browser Extensions"; RXSS Protect (2024)	Adv: dynamic symbolic execution detects DOM-sourced XSS; RXSS Protect targets reflected XSS in URLs. Dis: symbolic execution is expensive; RXSS Protect needs a separate ML server.	Source-to-sink analysis, reflected-XSS URL/payload analysis; add lightweight runtime DOM monitoring, local heuristics, risk-based warning/blocking and Playwright testing.

Literature Survey cont.

Feature	Existing Systems	Literature Survey	Advantages / Disadvantages	Proposed System
Browser Notification Scams	Chrome notification protection and Safe Browsing; AdGuard – Block Push API	Subramani et al., "When Push Comes to Ads: Measuring the Rise of (Malicious) Push Advertising" (2020)	Adv: analyzes notification abuse, origins, campaigns and landing URLs; shows malicious push ads can bypass ad blockers and URL filters. Dis: PushAdMiner needs instrumented browser infrastructure.	Notification-permission monitoring, page-behaviour analysis, URL reputation, selective warning/blocking and Playwright testing.
Browser Extension Abuse	Chrome Web Store / browser extension security	CPDetector: Automated Detection of Excessive Permissions in Chrome Extensions	Adv: static analysis finds excessive permissions by comparing requested permissions with actual API usage. Dis: mainly static; does not cover suspicious behaviour, page manipulation or script injection.	Static permission extraction, permission/API mismatch detection, over-privilege warnings and runtime security monitoring.
Cryptojacking	MinerBlock	"Analyzing In-browser Cryptojacking"	Adv: detects mining via WebSocket and mining-behaviour patterns; uses CPU-usage analysis. Dis: dynamic analysis can be resource-intensive in real time.	Lightweight CPU-threshold checks and WebAssembly / Web Worker detection.

Literature Survey cont.

Feature	Existing Systems	Literature Survey	Advantages / Disadvantages	Proposed System
Clickjacking	ClearClick	"Implementation of Extension for Browser to Detect Vulnerable Elements on Web Pages and Avoid Clickjacking"	Adv: detects deceptive elements through clickable-element and iframe analysis; examines visibility and cursor properties. Dis: may not detect all framing attacks.	Clickable-element, iframe, visibility/opacity, cursor, overlay-position and transparent-element analysis.
Email Phishing Detection	Email spam filters / Gmail spam protection	"A Machine Learning Approach for Phishing Email Detection Using Random Forest"	Adv: feature-based email analysis with Random Forest. Dis: conventional ML only; no RAG + LLM, URL-risk fusion or email-authentication signals.	Random Forest on URL count, suspicious URLs, length, urgency/credential keywords, HTML links and sender-domain mismatch; add SPF/DKIM/DMARC, RAG + LLM and URL-risk fusion.

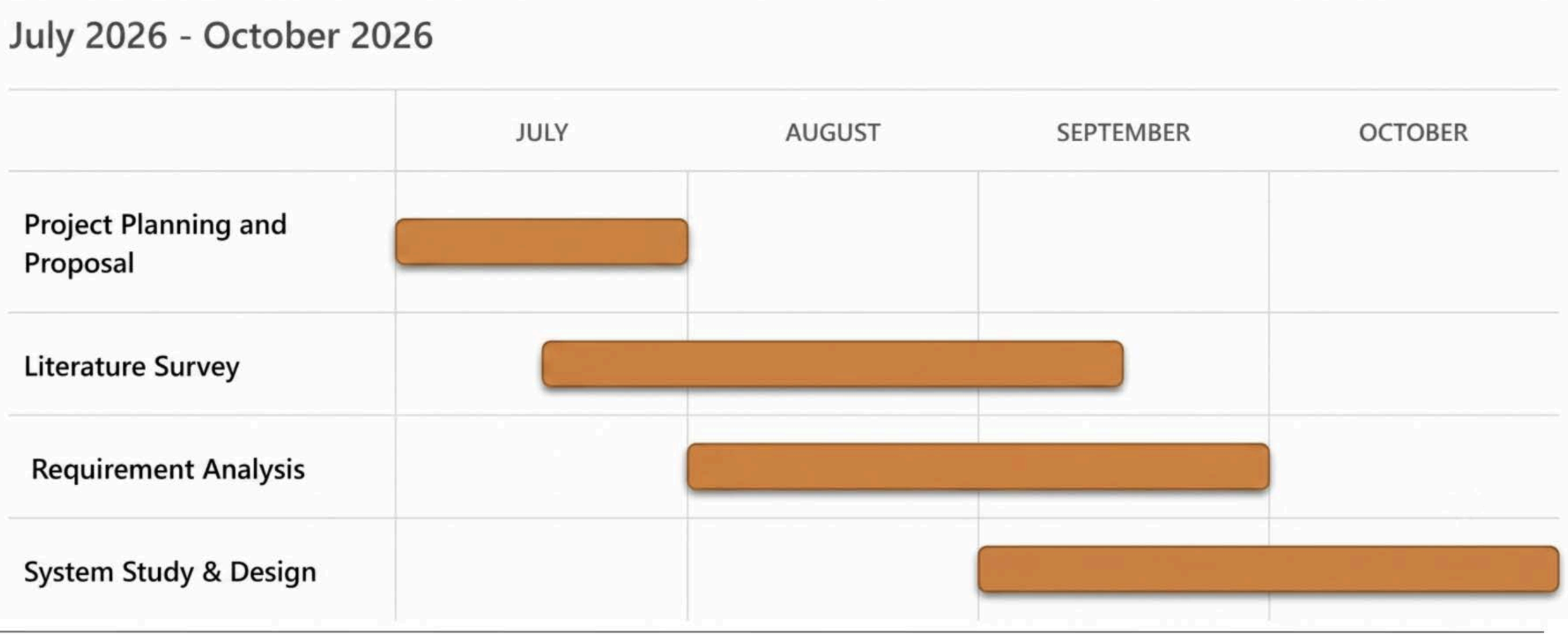
Literature Survey cont.

Feature	Existing Systems	Literature Survey	Advantages / Disadvantages	Proposed System
Malicious File Downloads	VirusTotal; antivirus / endpoint scanners	Šrndić & Laskov, "Hidost: A Static Machine-Learning-Based Detector of Malicious Files," EURASIP J. Inf. Secur. (2016)	Adv: static analysis without executing files; structural features; format-specific analysis. Dis: evaluated mainly on PDF and SWF, not EXE, DOC/DOCX or ZIP; uses Random Forest; no browser interception or threat-intelligence checks.	Format-specific static analysis of EXE/PDF/DOC/DOCX/ZIP, XGBoost, rules, hash and threat-intelligence checks, download holding, allow/block decisions with logging.
Credential Harvesting	Microsoft Defender SmartScreen; Microsoft Edge	Web scraping with Playwright (2026)	Adv: HTML and JavaScript can be evaluated with Playwright. Dis: not specified in the source document.	Use Playwright to detect external domains loading scripts and block scam pop-ups.

Software Requirements

Browser: Google Chrome and compatible Chromium-based browsers (Manifest V3 extension)
Frontend: React, Tailwind CSS
Backend: Python, FastAPI, Firebase Admin SDK (ID-token verification)
Authentication & Database: Firebase Authentication, MongoDB
ML/AI: XGBoost, Random Forest, SHAP, RAG + LLM pipeline
Analysis Tools: Playwright, QR-decoding components, file-analysis libraries, optional native helper for quarantine
External Services: Threat-intelligence and file-reputation services (providers to be finalized)

Gantt Chart


July 2026 - October 2026
JULY
AUGUST
SEPTEMBER
OCTOBER
Project Planning and Proposal
Literature Survey
Requirement Analysis
System Study & Design

Conclusion

- BrowseBlock brings a lightweight browser extension, a FastAPI backend and a React dashboard together as one unified, explainable security layer against ten categories of browser-based attacks.
- Two-stage analysis keeps real-time screening fast in the browser, while deeper XGBoost, RAG + LLM and static-analysis checks run on the backend.
- The design reports its limits honestly: a protective action is claimed only when actually applied, and incomplete analysis is never shown as safe.
- Next steps: finalize datasets, models and threat-intelligence providers, build the first prototype, and evaluate precision, recall and false-positive rates.

References

[1] N. Šrndić and P. Laskov, "Hidost: A static machine-learning-based detector of malicious files," *EURASIP J. Inf. Secur.*, 2016.

[2] Zeng et al., "Hidden Path: Understanding the intermediary in malicious redirections," 2022.

[3] Pan and Mao, "Detecting DOM-sourced cross-site scripting in browser extensions."

[4] "RXSS Protect," reflected XSS detection, 2024.

[5] Subramani et al., "When push comes to ads: Measuring the rise of (malicious) push advertising," 2020.

[6] "CPDetector: Automated detection of excessive permissions in Chrome extensions."

References cont.

[7] “Analyzing in-browser cryptojacking.”
[8] “Implementation of extension for browser to detect vulnerable elements on web pages and avoid clickjacking.”
[9] “A machine learning approach for phishing email detection using Random Forest.”
[10] “XGBoost-based URL phishing detection.”
[11] “Web scraping with Playwright,” 2026.